Reverse Polish Notation

AlgoExpert · Medium · Stacks

The problem

- Evaluate postfix tokens with + - * / (integer division truncating toward zero).

Intuition

- Operands go on a stack; each operator pops two and pushes the result.

Brute force vs optimal

- Build an expression tree then evaluate → extra structure
- Stack evaluation → O(n) time and space

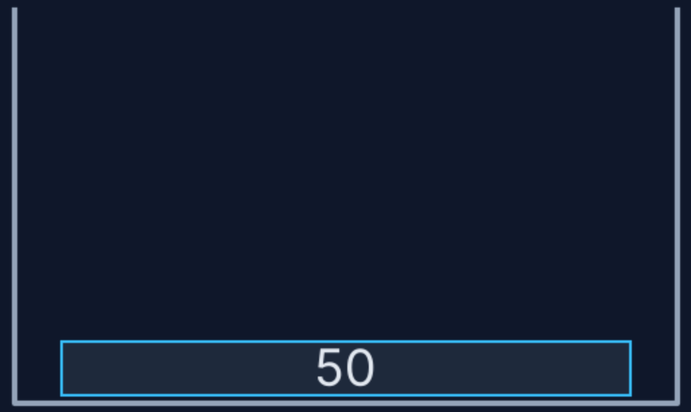
Approach

- 1. Push numbers
- 2. On an operator, pop right then left
- 3. Apply and push
- 4. Return the last value

Worked example



stack (top ↑)

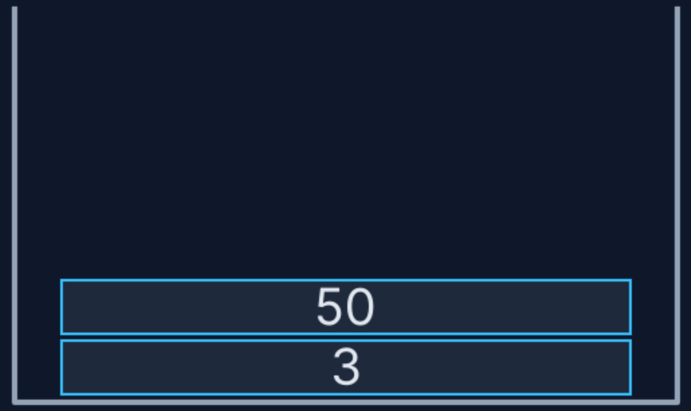


push 50

Worked example



stack (top ↑)

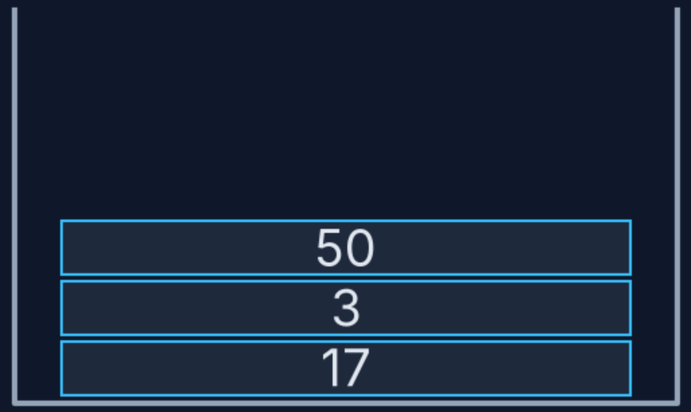


push 3

Worked example



stack (top ↑)

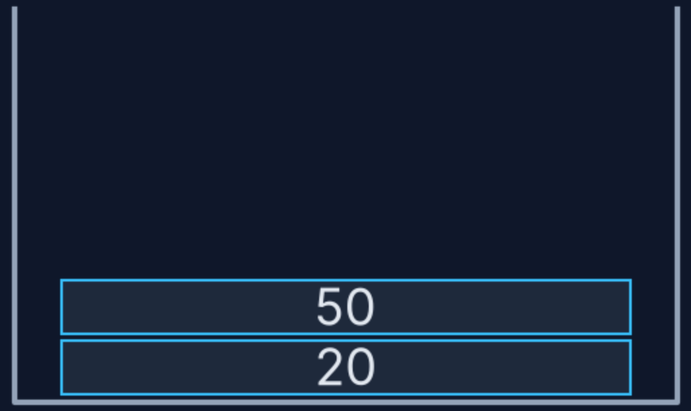


push 17

Worked example



stack (top ↑)

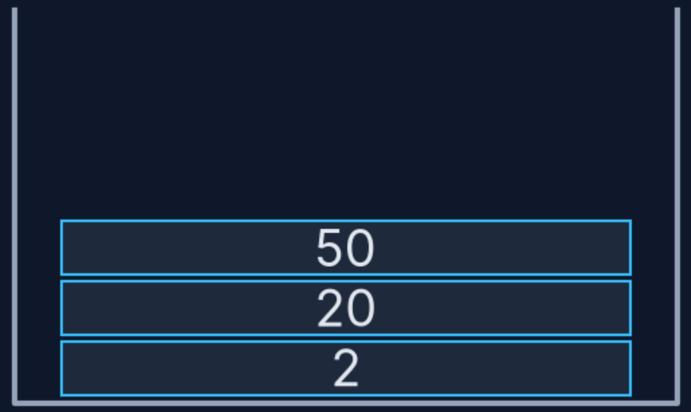


3 + 17 = 20

Worked example



stack (top ↑)

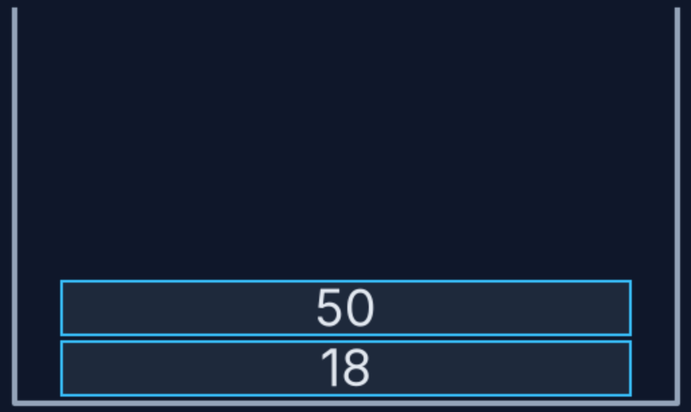


push 2

Worked example



stack (top ↑)

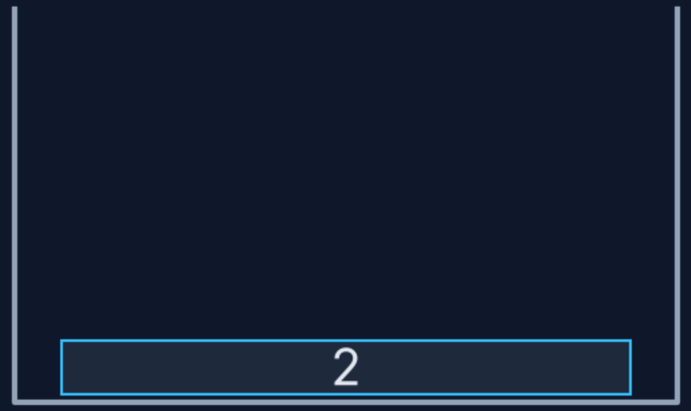


20 - 2 = 18

Worked example



stack (top ↑)



50 / 18 = 2

Java solution

```java
public class ReversePolishNotation {
    public static int reversePolishNotation(String[] tokens) {
        ArrayDeque<Integer> st = new ArrayDeque<>();
        for (String t : tokens) {
            switch (t) {
                case "+" -> st.push(st.pop() + st.pop());
                case "*" -> st.push(st.pop() * st.pop());
                case "-" -> { int b = st.pop(); st.push(st.pop() - b); }
                case "/" -> { int b = st.pop(); st.push(st.pop() / b); }
                default -> st.push(Integer.parseInt(t));
            }
        }
        return st.pop();
    }
}
```

Complexity

- Time: O(n)
- Space: O(n)

Edge cases & pitfalls

- Operand order for – and /: second pop is the left operand
- Integer division truncates toward zero
- Negative numbers are tokens too

Interview tips & follow-ups

- Follow-up: convert infix → RPN (shunting-yard)

Recap

- Operands go on a stack; each operator pops two and pushes the result.
- Complexity: O(n)